

# Capitolo 16 — Come leggere un processo WCM

|                    |                                                                                       |
|--------------------|---------------------------------------------------------------------------------------|
| <b>Pubblico</b>    | Lettori tecnici e non tecnici del WCM Process & Memory Book                           |
| <b>Versione</b>    | FROZEN-16                                                                             |
| <b>Data master</b> | 2026-08-29                                                                            |
| <b>Stato</b>       | FROZEN                                                                                |
| <b>Source path</b> | wcm/documentation/process-memory-book/<br>chapters/16_come_leggere_un_processo_wcm.md |
| <b>Source SHA</b>  | 7a26956                                                                               |
| <b>Release</b>     | 2026-08-29T23:33:20.223Z                                                              |

*GitHub main è la source of truth. Questo PDF è una release derivata: non costituisce approvazione né autorità.*

---

**Stato:** FROZEN **Parte:** VI — Il Libro dei Processi WCM **Scope:** WCM generale / domain-agnostic **Review:** Technical Review PASS / Human Comprehension Review PASS

---

## 16.0 Entriamo nel Process Book

Fin qui abbiamo imparato a orientarci nel WCM.

Abbiamo visto come una richiesta diventa lavoro governato, come INDEX-FIRST riduce il rumore, come Source Precedence distingue una fonte autorevole da una semplicemente disponibile e come il routing collega una situazione ai processi e ai protocolli applicabili.

Da questo capitolo cambiamo ancora prospettiva.

Non guarderemo più il Process Book soltanto dall'esterno.

Cominceremo a leggerlo dall'interno.

Il Process Register corrente contiene dodici processi. Ognuno governa un problema operativo differente: lifecycle di un Service Job, sincronizzazione del workspace, dispatch durevole, promozione dell'evidence, bootstrap del contesto, consolidamento della memoria, admission di un progetto, assurance della conoscenza, learning, continuità documentale, riconciliazione dello stato e closure di un WCM CHANGE.

Prima di affrontarli uno per uno serve però una grammatica comune.

**Leggere un processo WCM significa capire quale problema governa, quando entra in gioco, di quali informazioni ha bisogno, quali trasformazioni compie, quali vincoli incontra, che cosa produce e come sappiamo che ha davvero terminato il proprio lavoro.**

Questo capitolo non introduce un nuovo standard di processo.

Ricava una lente di lettura dai processi canonici correnti e dal Process Register, così da rendere i capitoli successivi comprensibili anche a chi non legge abitualmente specifiche tecniche.

---

## 16.1 Un processo non è una lista di istruzioni

Nel linguaggio quotidiano chiamiamo spesso "processo" una sequenza di passi.

Nel WCM la sequenza è importante, ma non basta.

Consideriamo due descrizioni.

La prima:

1. leggi il file
2. modifica il contenuto
3. salva

La seconda:

```
TRIGGER
→ INPUT AUTOREVOLI
→ VERIFICHE PRELIMINARI
→ TRANSIZIONI
→ GATE / DECISION POINT
→ OUTPUT
→ VERIFICA DI CHIUSURA
```

La seconda forma contiene qualcosa che la prima non contiene: **governance del movimento**.

Un processo WCM non dice soltanto cosa succede in condizioni ideali.

Deve permettere di capire anche quando deve partire, quando non deve partire, quali fonti deve usare, quali authority non può oltrepassare, cosa accade se manca un input o una verifica fallisce, quale stato deve essere persistito, quali altri processi o protocolli diventano applicabili, quale evidence dimostra che l'esito è reale e quando è corretto fermarsi.

Per questo un processo è meglio pensato come un **contratto operativo leggibile**. Non necessariamente come software o automazione, ma come descrizione governata di una trasformazione organizzativa.

---

## 16.2 La scheda mentale

Nei processi correnti le sezioni non sono tutte identiche e non devono esserlo per forza. PROC-001 espone molto chiaramente stati e regole del Service Job; PROC-005 enfatizza bootstrap, Resume Priority e Context Sufficiency Gate; PROC-006 rende centrali Impact Set e Consistency Bundle Check; PROC-010 organizza il proprio contenuto attorno a Documentation Impact Check e cross-document consistency.

Questa varietà è normale: processi diversi governano problemi diversi.

Possiamo però leggerli con una scheda mentale comune:

```
IDENTITÀ
↓
SCOPO
↓
TRIGGER
↓
INPUT
↓
FLUSSO / TRANSIZIONI
↓
GATE / DECISION POINT
↓
```

OUTPUT  
↓  
FAILURE MODE  
↓  
RELAZIONI  
↓  
EVIDENCE / MATURITY

Questa non è una nuova struttura obbligatoria del Process Book. È una **mappa pedagogica** per riconoscere gli elementi che ricorrono nella baseline corrente.

---

## 16.3 ID — l'identità del processo

Ogni processo canonico possiede un identificatore stabile: **PROC-001**, **PROC-005**, **PROC-010**, **PROC-012**. L'ID serve a distinguere il processo dal suo titolo e dalla sua descrizione e fornisce un riferimento compatto e stabile nelle relazioni con altri nodi.

Quando leggiamo l'ID chiediamoci: sto leggendo davvero il processo canonico corrente? L'ID coincide con quello del Process Register? Qual è lo status associato? Esistono riferimenti allo stesso ID in protocolli, decisioni o altri processi?

L'ID identifica. Non dimostra da solo che il documento sia corrente o applicabile. Per quello servono status, source precedence e scope.

---

## 16.4 Scopo — quale problema governa

Lo scopo risponde: **Perché questo processo esiste?** Impedisce di applicare un processo a un problema diverso solo perché alcuni passaggi sembrano simili.

PROC-001 governa il ciclo di vita persistente, verificabile e idempotente di un Service Job; PROC-005 ricostruisce il contesto operativo minimo e garantisce Resume Priority ai workflow incompleti; PROC-006 governa la continuità tra Working Memory e Persistent Organizational Memory dopo delta materiali.

STESSA FORMA DOCUMENTALE  
≠  
STESSO SCOPO

La domanda utile è: «**Uso questo processo quando devo governare...**». Se non riusciamo a completarla, non abbiamo ancora capito il processo.

---

## 16.5 Trigger — quando entra in gioco

Il trigger risponde: **Che cosa deve accadere perché questo processo diventi applicabile?** Può essere un evento, un cambiamento di stato, una nuova run, un delta materiale, una failure o una richiesta esplicita.

PROC-005 si applica quando una nuova sessione deve continuare lavoro precedente, quando un heartbeat riattiva un progetto, quando un agente deve ricostruire contesto o quando serve verificare authority e stato persistente. PROC-006 può essere attivato da una decisione, un cambio di stato, un output frozen, un apprendimento, un workflow checkpoint modificato o la chiusura di una sessione dopo lavoro materiale.

PROCESSO ESISTENTE  
≠  
PROCESSO ATTIVO IN QUESTO MOMENTO

A volte il trigger è esplicito, altre deve essere derivato dal significato dell'operazione. Il trigger può quindi richiedere reasoning; una volta riconosciuto, le conseguenze strutturate possono essere molto più deterministiche.

## 16.6 Input — di cosa ha bisogno

Un processo non opera nel vuoto. Gli input sono le informazioni, gli stati, i documenti o i riferimenti necessari perché il processo possa compiere la propria trasformazione.

PROC-005 dichiara, tra gli input, Working Memory, ruolo, goal/task, entry point generale, stato del progetto, workflow checkpoint e indici attivi. PROC-010 usa fonti differenti a seconda che descriva WCM generale o un progetto: governance, capabilities, architecture, Process Register, runtime, state, project KB, automazioni ed evidence pertinenti.

INPUT NECESSARI  
≠  
TUTTO CIÒ CHE POSSIAMO LEGGERE

Qui ritornano INDEX-FIRST e Context Sufficiency. Due file possono contenere informazioni sullo stesso argomento, ma non avere la stessa funzione. Per execution facts il runtime strutturato può prevalere sulla human view; per una regola di metodo la fonte canonica del processo o protocollo ha una funzione differente dalla sua spiegazione in un manuale.

## 16.7 Flusso — che trasformazione compie

Il flusso descrive il movimento. PROC-001 espone:

HOLD  
→ READY  
→ IN\_PROGRESS  
→ DONE

con stati alternativi per blocco, failure o cancellazione.

PROC-006 usa:

DELTA DETECTION  
→ CLASSIFICATION  
→ AUTHORITY / STATUS CHECK  
→ CAUSAL IMPACT CHECK  
→ IMPACT SET  
→ CONSOLIDATION

Un flusso può descrivere stati, attività, verifiche, trasformazioni di dati, transizioni di workflow o loop di controllo. Per ogni passaggio chiediamoci: **«Che cosa cambia tra prima e dopo?»**.

## 16.8 Gate e decision point — dove il flusso può cambiare

Un processo può contenere biforcazioni:

```
CHECK
├─ PASS → CONTINUE
└─ FAIL → STOP / REPAIR / ESCALATE
```

Un gate è un punto in cui una condizione deve essere soddisfatta prima di poter procedere. Un decision point è un punto in cui il percorso dipende da un fatto, uno stato o una decisione.

Nel bootstrap di PROC-005 troviamo Resume Priority Gate e Context Sufficiency Gate. Nel Documentation Continuity Loop troviamo il Documentation Impact Check. Non tutti i gate chiedono una decisione umana: un gate può essere deterministico (**schema valido?**) oppure richiedere authority (**WCM CHANGE autorizzato?**). Confonderli significa chiedere all'umano decisioni meccaniche o automatizzare decisioni che appartengono all'authority.

## 16.9 Output — cosa deve esistere dopo

L'output risponde: **Che cosa deve essere vero o disponibile se il processo ha funzionato?** Non sempre è un file.

PROC-005 può produrre un contesto operativo sufficientemente ricostruito. PROC-001 richiede alla chiusura almeno stato finale, sintesi, output, evidence/test, acceptance criteria, limiti residui e altri elementi. PROC-006 produce memoria persistente coerente e consistency bundle verde.

```
OUTPUT
≠
SEMPRE DOCUMENTO
```

Un output può essere stato, artefatto, record, checkpoint, decisione di routing, insieme di evidenze, vista riconciliata o condizione verificata.

```
ARTEFATTO PRODOTTO
≠
PROCESSO COMPLETATO
```

## 16.10 Failure mode — come può fallire

Una specifica utile descrive anche gli errori tipici. PROC-005 include ignorare un workflow incompleto, affidarsi alla chat, rieseguire step completati, leggere tutto il repository per abitudine o continuare retrieval oltre la sufficienza. PROC-006 include copiare tutta la chat nella KB, trasformare una proposta in decisione, aggiornare un solo file ignorando dipendenze, dimenticare il workflow checkpoint o dichiarare **COMPLETED** con consistency bundle non verde.

I failure mode mostrano il **confine negativo del processo**: cosa il processo esiste per impedire. Possono essere tecnici, cognitivi o organizzativi.

## 16.11 Relazioni — il processo non vive da solo

Nel WCM un processo è un nodo della memoria organizzativa e quasi sempre dipende da altri nodi. PROC-005 è collegato, tra gli altri, a DEC-012, PROT-009, PROT-005, PROC-006, PROT-007, CONCEPT-007 e CONCEPT-008. PROC-006 è collegato a workflow execution, decision impact, knowledge health, assurance e change closure.

**un processo raramente contiene da solo tutte le regole necessarie alla propria esecuzione.**

Il processo descrive il flusso principale; protocolli, decisioni, concept, template e runtime completano il contesto quando necessari.

RELATIONSHIP  
→ NAVIGATION OPTION

NON

RELATIONSHIP  
→ FULL RELOAD OBBLIGATORIO

## 16.12 Processi e protocolli: non confonderli

Un processo governa principalmente un flusso verso un risultato. Un protocollo governa principalmente una regola, un vincolo, una guard o una disciplina applicabile a uno o più flussi.

PROC-005  
Agent-Ready Context Bootstrap

PROT-005  
Index-First Progressive Retrieval

Il processo risponde «Come ricostruisco il contesto operativo necessario?». Il protocollo risponde «Come devo recuperare la conoscenza senza leggere indiscriminatamente tutto?». PROC-005 usa PROT-005, ma PROT-005 può essere utile anche fuori da PROC-005.

## 16.13 Status — quanto è maturo

Nel Process Register corrente troviamo status come **VALIDATED**, **VALIDATED BY GOVERNANCE**, **FIELD VALIDATION IN PROGRESS**, **ACTIVE / FIELD VALIDATION**, **FIRST FIELD VALIDATION**.

Lo status serve a capire che tipo di affermazione possiamo fare sul processo. **VALIDATED** non significa efficacia universale in ogni organizzazione, dominio, scala o configurazione tecnica. **FIELD VALIDATION** indica uso/osservazione sul campo con maturità empirica ancora in crescita. **ACTIVE** indica appartenenza alla baseline corrente nel proprio scope.

È BASELINE CORRENTE?

quanto è MATURO / VALIDATO?

Non sono la stessa domanda.

## 16.14 Evidence — perché crediamo che funzioni

Alcuni processi includono evidence o razionale. PROC-001 richiama POC e successive esecuzioni sul lifecycle e durable dispatch; PROC-005 richiama authority e failure reali che hanno motivato Session-Independent Workflow Execution e Resume Priority.

EVIDENCE

≠

BASELINE AUTOMATICA

L'evidence può sostenere decisione, validazione o promozione; non acquisisce da sola authority normativa.

## 16.15 Authority — chi rende valido il processo

Alcuni processi dichiarano esplicitamente un campo **Authority**; altri si affidano al Process Register e alla governance corrente. L'authority risponde: **Chi o quale decisione rende questa regola parte del metodo corrente?** PROC-005 dichiara DEC-012; PROC-010 dichiara DEC-010 + DEC-014. L'authority aiuta a ricostruire lineage e forza normativa e non va dedotta dal fatto che il documento sia recente o ben scritto.

## 16.16 Owner — chi presidia il processo

Molti processi dichiarano un owner. L'owner può indicare chi presidia, esegue o mantiene il processo nel perimetro definito, ma non equivale ad authority illimitata di modifica.

PROCESS OWNER

≠

CHANGE AUTHORITY ILLIMITATA

---

## 16.17 Un processo può essere human, cognitive, deterministic o ibrido

Il WCM non assume che ogni processo debba essere automatizzato nello stesso modo. Può contenere parti cognitive, deterministiche, human-gated, service-driven o ibride. PROC-005 contiene reasoning sulla pertinenza e checkpoint strutturati; PROC-006 richiede classificazione semantica del delta ma può usare verifiche strutturali deterministiche; PROC-010 combina documentation impact e pipeline distributiva.

La domanda è: **Quale parte richiede comprensione e quale parte può essere enforcement meccanico?**

---

## 16.18 Persistenza — che cosa deve sopravvivere

Molti processi producono effetti che devono sopravvivere alla sessione: stato del job, workflow checkpoint, decision record, evidence, baseline promossa, living ledger, documentation master, derived state, change manifest. Quando un processo modifica persistent state chiediamoci: **Qual è il writer autorizzato e qual è la source of truth?** Le regole trasversali vengono ereditate attraverso routing e protocolli quando applicabili.

---

## 16.19 Completion — quando possiamo dire «finito»

PROC-001 ammette **DONE** solo dopo acceptance; PROC-005 termina quando il Context Sufficiency Gate è soddisfatto; PROC-006 termina con Consistency Bundle Verification verde; PROC-010 è PASS quando Documentation Impact Check è esplicito, ogni YES è riflesso nei master, non restano conflitti nascosti e l'eventuale release è generata e verificata.

ULTIMO OUTPUT VISIBILE  
≠  
COMPLETION AUTOMATICA

---

## 16.20 Un esempio di lettura: PROC-005

**Identità:** PROC-005 — Agent-Ready Context Bootstrap.

**Scopo:** ricostruire il contesto operativo minimo e riprendere workflow incompleti prima di cercare nuovo lavoro.

**Trigger:** nuova sessione, heartbeat, ripresa progetto, onboarding agente/service, verifica stato/authority.



**Input:** Working Memory, ruolo, goal/task, entry point, stato persistente, workflow checkpoint, indici attivi.

**Flusso:**

```
WORKING MEMORY
→ PROJECT / STATE
→ RESUME CHECK
→ INDEX-FIRST RETRIEVAL
→ CONTEXT SUFFICIENCY
→ ACT / RESUME
```

**Gate:** Resume Priority Gate; Context Sufficiency Gate; WAITING\_AUTHORITY quando applicabile.

**Output:** contesto operativo sufficiente con authority, scope, next transition e true stop comprese.

**Failure mode:** ricostruire da chat, duplicare step, leggere tutto, usare raw prima della baseline, continuare retrieval senza motivo.

**Relazioni:** PROT-009, PROT-005, PROC-006, PROT-007 e concept Dual Memory / Agent-Ready Architecture.

**Maturity:** Validated by Governance, evoluzione session-independent attiva, field validation in progress.

---

## 16.21 Un secondo esempio: PROC-001

**Identità:** PROC-001 – Service Job Lifecycle.

**Scopo:** governare lifecycle persistente, verificabile e idempotente di un Service Job.

**Trigger:** creazione, attivazione, presa in carico, blocco o chiusura.

**Flusso:** HOLD → READY → IN\_PROGRESS → DONE, con BLOCKED\_LOCAL, BLOCKED\_WISE, FAILED, CANCELLED.

**Gate:** HOLD non eseguibile; prima di IN\_PROGRESS verificare contratto, branch, workspace, input, strumenti e limiti; DONE soltanto dopo acceptance verificata.

**Output:** stato finale, sintesi, output, evidence/test, acceptance criteria, limiti residui e altri elementi di closure.

**Relazioni:** quando attivato da control plane periodico entrano PROC-003 e PROT-004.

**Maturity:** VALIDATED, con evidence da POC e successive esecuzioni.

---

## 16.22 Cosa non fare leggendo un processo

Anti-pattern: leggere solo il diagramma; leggere solo regole numerate; confondere titolo e scope; saltare lo status; ignorare relazioni; trattare evidence come authority; assumere output = completion; trasformare la spiegazione editoriale in nuova regola.

Il libro spiega la baseline. Non la modifica.

---

## 16.23 La scheda compatta del lettore

1. ID – quale processo sto leggendo?
2. SCOPO – quale problema governa?
3. TRIGGER – quando entra in gioco?
4. INPUT – di cosa ha bisogno e da quali fonti?
5. FLUSSO – che trasformazione compie?
6. GATE – dove può continuare, deviare o fermarsi?
7. OUTPUT – cosa deve esistere dopo?
8. FAILURE MODE – quali errori deve impedire?
9. RELAZIONI – quali altri nodi lo governano o lo completano?
10. EVIDENCE / MATURITY – quanto è validato e su quali basi?

A queste aggiungiamo: **QUAL È L'AUTHORITY?** e **QUAL È LA VERA CONDIZIONE DI COMPLETION?**. Non sono nuovi campi obbligatori, sono domande di lettura.

---

## 16.24 Cosa abbiamo ottenuto

Ora possiamo entrare nel Process Book senza trattarlo come un catalogo di procedure da memorizzare.

```
PROBLEMA
↓
TRIGGER
↓
INPUT AUTOREVOLI
↓
TRASFORMAZIONE
↓
GATE
↓
OUTPUT
↓
VERIFICA
↓
CONTINUITÀ
```

E possiamo guardarlo da più prospettive:

```
SEMANTICA
+ GOVERNANCE
+ EXECUTION
+ PERSISTENZA
+ EVIDENCE
+ MATURITY
```

Dal prossimo capitolo cominceremo il percorso processo per processo. Il primo sarà **PROC-001 — Service Job Lifecycle**.

---

## Source Map

## Fonti canoniche principali

- `WCM_AGENT_START.md` — bootstrap, Source Precedence, Resume Priority, completion e persistent workflow invariants;
- `wcm/process-book/PROCESS_REGISTER.md` — baseline corrente dei 12 processi, status, scopi e relazioni principali;
- `wcm/process-book/processes/PROC-001_SERVICE_JOB_LIFECYCLE.md` — esempio di processo lifecycle con stati, regole, closure ed evidence;
- `wcm/process-book/processes/PROC-005_AGENT_READY_CONTEXT_BOOTSTRAP.md` — esempio di processo con Obiettivo, Trigger, Input, Processo, Gate, Output, Failure mode, Evidence e Relazioni;
- `wcm/process-book/processes/PROC-006_MEMORY_CONSOLIDATION_LOOP.md` — esempio di processo con Impact Set, Consistency Bundle e Completion dependency;
- `wcm/process-book/processes/PROC-010_DOCUMENTATION_CONTINUITY_LOOP.md` — esempio di processo con Documentation Impact Check, cross-document consistency e release closure;
- `wcm/process-book/templates/` — template correnti del Process Book; la baseline non espone un unico template obbligatorio generale per l'anatomia di tutti i processi.

## Relazioni

```
CH16
├ CONTINUES → CH13 / CH14 / CH15
├ MAPS → PROCESS_REGISTER
├ EXPLAINS_HOW_TO_READ → PROC-001 ... PROC-012
├ USES_AS_EXAMPLES → PROC-001 / PROC-005 / PROC-006 / PROC-010
└ PREPARES → CH17 — PROC-001 Service Job Lifecycle
```

## Maturity note

La scheda di lettura presentata in questo capitolo è una sintesi pedagogica ricavata dalla baseline corrente, non un nuovo schema canonico imposto ai processi WCM. I dodici processi hanno livelli di maturità differenti, esplicitati nel Process Register e nei rispettivi master. **VALIDATED**, **ACTIVE** e **FIELD VALIDATION** devono essere letti nel loro scope: non costituiscono una dimostrazione di validità universale in ogni dominio o organizzazione. Il libro descrive il metodo corrente senza promuovere nuove regole.